

```
In [1]: import numpy as np
import pandas as pd
from sklearn.linear_model import LinearRegression
```

```
In [2]: df = pd.read_csv("Advertising.csv").iloc[:,1:]
df.head()
```

```
Out[2]:
```

|   | TV    | Radio | Newspaper | Sales |
|---|-------|-------|-----------|-------|
| 0 | 230.1 | 37.8  | 69.2      | 22.1  |
| 1 | 44.5  | 39.3  | 45.1      | 10.4  |
| 2 | 17.2  | 45.9  | 69.3      | 9.3   |
| 3 | 151.5 | 41.3  | 58.5      | 18.5  |
| 4 | 180.8 | 10.8  | 58.4      | 12.9  |

```
In [7]: # 2 variables: output should match the numbers output by LinearRegression class (ab
class myLinearModel:
    def __init__(self): #initialize attributes
        self.slope1 = 0.0
        self.intercept = 0.0
        self.alpha = 0.1

    def fit(self, X, y): # self = refers to the slope and intercept in __init__(sel

        # Standardize data
        x_standard = (X - np.mean(X, axis = 0))/np.std(X, axis = 0) # z-score to st
        y_standard = (y - np.mean(y, axis = 0))/np.std(y, axis = 0)

        # Normal distribution
        x_bias = x_standard.reshape(-1, 1)
        x_bias = np.c_[x_bias, np.ones(len(x_bias))] # Add column of 1's
        x_t = np.transpose(x_bias) # x transpose
        w = np.zeros((2, 1))
        y_bias = y_standard.reshape(-1, 1)
        #print("X : ", X.shape, "; x_standard: ", x_standard.shape, "; x_bias: ", x
        min_var = x_t @ ((x_bias @ w) - y_bias)

        # Gradient descent
        for i in range(5000):
            predictions = x_bias @ w
            error = predictions - y_bias
            gradient = (2 / len(X)) * (x_t @ error) # equivalent to partial derivat
            w = w - self.alpha * gradient # update weights

        w_standardized, b_standardized = w[0].item(), w[1].item()
        # print("Standardized m1, b:", w)

        # Reverse calculate true values
        # Obtain mean and standard deviation
        x_mean = np.mean(X, axis=0)
```

```

x_std = np.std(X, axis=0)
y_mean = np.mean(y)
y_std = np.std(y)

w_standardized, b_standardized = w

# Reverse calculate true m values (m1, m2, m3)
true_m = w_standardized * (y_std / x_std)

# Reverse calculate true b
adjustment = np.sum(true_m * x_mean) # true_m encapsulates w_standardized *
true_b = y_mean + (y_std * b_standardized) - adjustment

print("True Weights (m1):", true_m)
print("True Intercept (b):", true_b)

def score(self, X, y):
    pass
    # return

```

```

In [8]: x = df.iloc[:10,0].values.reshape(-1,1)
        y = df.iloc[:10,-1].values.reshape(-1,1)

        myModel = myLinearModel()
        myModel.fit(x, y)

```

```

True Weights (m1): [0.04657946]
True Intercept (b): [7.33401842]

```

```

In [5]: # All variables in dataset
class myLinearModel:
    def __init__(self): #initialize attributes
        self.slope1 = 0.0
        self.slope2 = 0.0
        self.slope3 = 0.0
        self.intercept = 0.0
        self.alpha = 0.1

    def fit(self, X, y): # self = refers to the slope and intercept in __init__(sel

        # Standardize data
        x_standard = (X - np.mean(X, axis = 0))/np.std(X, axis = 0) # z-score to st
        y_standard = (y - np.mean(y, axis = 0))/np.std(y, axis = 0)

        # Normal distribution
        x_bias = x_standard
        x_bias = np.c_[x_bias, np.ones(len(x_bias))] # Add column of 1's
        x_t = np.transpose(x_bias) # x transpose
        w = np.zeros((4, 1))
        y_bias = y_standard.reshape(len(X), 1)
        # print("X : ", X.shape, "; x_standard: ", x_standard.shape, "; x_bias: ",
        min_var = x_t @ ((x_bias @ w) - y_bias)

        # Gradient descent
        for i in range(5000):
            predictions = x_bias @ w

```

```

        error = predictions - y_bias
        gradient = (2 / len(X)) * (x_t @ error) # equivalent to partial derivat
        w = w - self.alpha * gradient # update weights

w_standardized, b_standardized = w[0:3].flatten(), w[3].item()
# print(w_standardized, b_standardized)

# Reverse calculate true values
# Obtain mean and standard deviation
x_mean = np.mean(X, axis=0)
x_std = np.std(X, axis=0)
y_mean = np.mean(y)
y_std = np.std(y)

# Reverse calculate true m values (m1, m2, m3)
true_m = w_standardized * (y_std / x_std)

# Reverse calculate true b
adjustment = np.sum(true_m * x_mean) # true_m encapsulates w_standardized *
true_b = y_mean + (y_std * b_standardized) - adjustment

print("True Weights (m1, m2, m3):", true_m)
print("True Intercept (b):", true_b)

def score(self, X, y):
    pass
    # return

```

```

In [6]: x = df.iloc[:10,0:3].values
        y = df.iloc[:10,-1].values

        myModel = myLinearModel()
        myModel.fit(x, y)

```

```

True Weights (m1, m2, m3): [ 0.06334544  0.23743789 -0.06710096]
True Intercept (b): 1.8554832371820318

```

```

In [ ]:

```